



**CENTRO UNIVERSITÁRIO ESTÁCIO DE SÁ
CAMPUS TERESINA**

Data de entrega: 15 de junho de 2026

SISTEMA DE CADASTRO DE PRODUTOS

Disciplina: Programação de Software Básico em C

Turma: 2026.1

Gabriel de Oliveira Barbosa
Jadson Gregório de Araújo Costa Soares
Nuno Rafael de Moura Bezerra

Prof. Douglas Lopes de Sousa Mendes

**Teresina – PI
15 de junho 2026**

Sumário

1	INTRODUÇÃO	2
1.1	Qual problema o programa procura resolver?	2
1.2	Qual é a finalidade do cadastro de produtos?	2
1.3	Quais conteúdos da linguagem C foram utilizados?	2
2	DESCRIÇÃO DO FUNCIONAMENTO	3
2.1	Como a quantidade de produtos é informada?	3
2.2	Como a memória é alocada?	3
2.3	Como os produtos são cadastrados?	3
2.4	Como os valores do estoque são calculados?	3
2.5	Como o produto mais caro é identificado?	4
2.6	Como a memória é liberada?	4
3	ESTRUTURA UTILIZADA	4
4	QUESTÕES CONCEITUAIS	5
4.1	O que é um ponteiro na linguagem C?	5
4.2	Por que a função <i>free()</i> deve ser utilizada após uma alocação dinâmica de memória?	5
4.3	Para que a biblioteca <i>OpenGL</i> pode ser utilizada?	5
4.4	Explique, de maneira simples, a diferença entre um processo e uma thread.	5
5	CÓDIGO FONTE E EXECUÇÃO	6
5.1	Código fonte	6
5.2	Evidências de execução	9
6	CONCLUSÃO	11

1 INTRODUÇÃO

1.1 Qual problema o programa procura resolver?

O programa foi desenvolvido para resolver o problema do controle básico de estoque de produtos em uma empresa, loja ou estabelecimento comercial. Ele permite registrar informações importantes sobre cada produto, como código, nome, preço e quantidade disponível. Dessa forma, os dados ficam organizados e podem ser consultados posteriormente de maneira simples e rápida.

Além do cadastro, o sistema também auxilia no gerenciamento do estoque ao calcular automaticamente o valor total armazenado em produtos. Isso permite que o usuário tenha uma visão geral do capital investido no estoque. O programa ainda identifica qual produto possui o maior preço entre os cadastrados, fornecendo informações úteis para análises e tomadas de decisão relacionadas ao negócio.

1.2 Qual é a finalidade do cadastro de produtos?

A finalidade do cadastro de produtos é armazenar informações essenciais sobre os itens que compõem o estoque da empresa. Cada produto recebe um código de identificação, um nome, um preço e uma quantidade disponível, permitindo um controle mais eficiente dos recursos disponíveis para venda ou uso.

Com essas informações registradas, o usuário pode consultar os produtos cadastrados sempre que necessário. Isso facilita a organização do estoque, reduz a possibilidade de erros e melhora o acompanhamento dos produtos existentes. Além disso, o cadastro serve como base para os cálculos realizados pelo sistema, como o valor total do estoque e a identificação do produto mais caro.

1.3 Quais conteúdos da linguagem C foram utilizados?

O programa utiliza diversos conceitos fundamentais da linguagem C. Entre eles estão estruturas (*struct*), funções, vetores, ponteiros, alocação dinâmica de memória, entrada e saída de dados com *scanf* e *printf*, além de bibliotecas como *stdio.h*, *stdlib.h* e *string.h*.

Também são utilizados laços de repetição *for*, estruturas condicionais *if*, manipulação de strings com *fgets* e *strcspn*, além de operações matemáticas para calcular valores do estoque. Esses recursos tornam o programa mais organizado e eficiente.

2 DESCRIÇÃO DO FUNCIONAMENTO

2.1 Como a quantidade de produtos é informada?

A quantidade de produtos é informada pelo próprio usuário no início da execução do programa. O sistema solicita que seja digitado um valor entre 1 e 10, representando quantos produtos serão cadastrados. Após a entrada do valor por meio do comando *scanf*, o programa verifica se a quantidade informada está dentro do intervalo permitido. Caso o número seja inválido, uma mensagem de erro é exibida e o programa é encerrado.

2.2 Como a memória é alocada?

A memória é alocada dinamicamente utilizando a função *malloc()*. Essa função reserva espaço suficiente para armazenar a quantidade de produtos informada pelo usuário durante a execução do programa. A instrução utilizada é:

```
produtos = malloc(quantidade * sizeof(struct Produto));
```

Dessa forma, a quantidade de memória reservada depende diretamente do número de produtos cadastrados, evitando desperdícios de recursos.

2.3 Como os produtos são cadastrados?

Os produtos são cadastrados por meio da função *cadaststrarProdutos()*. Nessa função, um laço de repetição percorre todas as posições do vetor de produtos e solicita as informações necessárias para cada item. Para cada produto, o usuário informa o código, o nome, o preço e a quantidade disponível. Esses dados são armazenados nos campos correspondentes da estrutura *Produto*, formando assim o cadastro completo do estoque.

2.4 Como os valores do estoque são calculados?

O valor em estoque de cada produto é calculado multiplicando o preço unitário pela quantidade disponível. Esse cálculo é realizado na função *listarProdutos()*. A fórmula utilizada é:

```
valorEstoque = produtos[i].preco * produtos[i].quantidade;
```

Já o valor total do estoque é obtido somando os valores de todos os produtos cadastrados, sendo calculado na função *exibirResumo()*.

2.5 Como o produto mais caro é identificado?

O produto mais caro é identificado durante a execução da função *exibirResumo()*. Inicialmente, o programa considera o primeiro produto como o mais caro e armazena seu índice e preço. Em seguida, um laço percorre todos os produtos, comparando seus preços. Sempre que encontra um produto com preço superior ao atual, o programa atualiza as variáveis responsáveis por armazenar o maior preço e a posição do produto mais caro.

2.6 Como a memória é liberada?

Após o término de todas as operações, a memória utilizada é liberada com a função *free()*. Essa prática é importante para evitar desperdício de memória e possíveis vazamentos durante a execução do programa. A instrução utilizada é:

```
free(produtos);  
produtos = NULL;
```

A memória reservada é devolvida ao sistema operacional. Em seguida, o ponteiro recebe o valor *NULL*, evitando que ele continue apontando para uma região de memória que já foi liberada.

3 ESTRUTURA UTILIZADA

Definida abaixo está a struct, estrutura utilizada referentes ao armazenamento dos dados dos produtos em questão no projeto, sua definição foi separada em 4 campos para melhor explicar as funções de cada.

```
struct Produto{  
    int codigo;  
    char nome[50];  
    float preco;  
    int quantidade;  
};
```

- 1 - Código: O código único para separar os itens, serve para identificação de cada qual, além do nome do produto.
- 2 - Nome: No formato da variável 'char', limitada em até 49 caracteres fora a *string* especial \0.

- 3 - Preço: Se refere aos valores que serão atribuídos a cada um dos produtos, no formato de ponto flutuante float, justamente para suportar a inclusão de casas decimais.
- 4 - Quantidade: Se refere ao volume de itens que serão adicionados pelo usuário do sistema de cadastro para cada um dos produtos, pode ser usado como meio de definir o número de itens do dado produto em questão.
- 5 - Produto: Já esse termo é utilizado para organizar de forma mais lógica e direta as informações atreladas a ele, é a definição da dada estrutura.

4 QUESTÕES CONCEITUAIS

4.1 O que é um ponteiro na linguagem C?

Um ponteiro é uma variável especial que armazena o endereço de memória de outra variável. Por meio dos ponteiros, é possível acessar e modificar valores armazenados em diferentes posições da memória, além de permitir a manipulação eficiente de estruturas de dados e a alocação dinâmica de memória.

4.2 Por que a função *free()* deve ser utilizada após uma alocação dinâmica de memória?

A função *free()* é utilizada para liberar a memória previamente alocada dinamicamente por funções como *malloc()*, *calloc()* ou *realloc()*. Seu uso é importante para evitar vazamentos de memória (*memory leaks*), que ocorrem quando áreas de memória deixam de ser utilizadas pelo programa, mas continuam ocupadas até o encerramento da execução.

4.3 Para que a biblioteca *OpenGL* pode ser utilizada?

A *OpenGL* (*Open Graphics Library*) é uma biblioteca gráfica utilizada para o desenvolvimento de aplicações que realizam renderização de gráficos em duas e três dimensões. Ela é amplamente empregada na criação de jogos, simuladores, sistemas de visualização científica e aplicações que necessitam de processamento gráfico acelerado.

4.4 Explique, de maneira simples, a diferença entre um processo e uma thread.

Um processo é uma instância de um programa em execução, possuindo seu próprio espaço de memória e recursos do sistema operacional.

Já uma thread é uma unidade de execução dentro de um processo. Diferentes threads pertencentes ao mesmo processo compartilham recursos e memória, permitindo a execução simultânea de múltiplas tarefas de forma mais eficiente.

5 CÓDIGO FONTE E EXECUÇÃO

5.1 Código fonte

Abaixo está transcrito e comentado o código utilizado para o projeto. Vale lembrar que ele adiciona até 10 produtos, e em relação aos preços, é necessários os por em formato 'x.x', onde x representa os possíveis números a serem postos no cadastro, por conta da leitura dos decimais *float* que apenas suporta o símbolo do ponto ao invés da virgula. O código a seguir:

[Código do projeto em C]

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 // Estrutura para armazenar dados de um produto
6 struct Produto{
7     int codigo;
8     char nome[50];
9     float preco;
10    int quantidade;
11 };
12 // Funcao para cadastrar produtos
13 void cadastrarProdutos(struct Produto *produtos, int quantidade){
14     for (int i = 0; i < quantidade; i++){
15
16         printf("\n-----\n");
17         printf("CADASTRO DO PRODUTO %d\n", i + 1);
18         printf("-----\n");
19         printf("Codigo: ");
20         scanf("%d", &produtos[i].codigo);
21
22         getchar();
23         // Remove o '\n' deixado pelo scanf
24
25         printf("Nome: ");
26         fgets(produtos[i].nome, sizeof(produtos[i].nome), stdin);
```

```

27
28 // Remove o '\n' do final da string
29 produtos[i].nome[strcspn(produtos[i].nome, "\n")] = '\0';
30
31 printf("Pre o: ");
32 scanf("%f", &produtos[i].preco);
33 printf("Quantidade dispon vel: ");
34 scanf("%d", &produtos[i].quantidade);
35 }
36 }
37 void listarProdutos(struct Produto *produtos, int quantidade){\\
38 printf("\n\n-----\n");\\
39 printf("PRODUTOS CADASTRADOS\n");\\
40 printf("-----\n");\\
41
42 for (int i = 0; i < quantidade; i++){
43 float valorEstoque = produtos[i].preco * produtos[i].quantidade;
44
45 printf("\n----- PRODUTO %d -----\n", i + 1);
46 printf("C digo: %d\n", produtos[i].codigo);
47 printf("Nome: %s\n", produtos[i].nome);
48 printf("Pre o: R$ %.2f\n", produtos[i].preco);
49 printf("Quantidade: %d\n", produtos[i].quantidade);
50 printf("Valor em estoque: R$ %.2f\n", valorEstoque);
51 }
52 }
53 // Funcao para exibir resumo
54 void exibirResumo(struct Produto *produtos, int quantidade){
55 float valorTotal = 0.0;
56 float maiorPreco = produtos[0].preco;
57 int indiceMaisCaro = 0;
58 for (int i = 0; i < quantidade; i++){\\
59 valorTotal += produtos[i].preco * produtos[i].quantidade;
60
61 if (produtos[i].preco > maiorPreco){
62 maiorPreco = produtos[i].preco;
63 indiceMaisCaro = i;
64 }
65 }
66 printf("\n\n-----\n");
67 printf("RESUMO DO ESTOQUE\n");

```



```

68 printf("-----\n");
69 printf("Valor total do estoque: R$ %.2f\n", valorTotal);
70 printf("Produto mais caro sera: %s\n", produtos[indiceMaisCaro].
    nome);
71 printf("O preço do produto mais caro: R$ %.2f\n", maiorPreco);
72 }
73 int main(){
74     struct Produto *produtos;
75     int quantidade;
76
77     printf("-----\n");
78     printf("SISTEMA DE CADASTRO DE PRODUTOS\n");
79     printf("-----\n");
80     printf("Quantos produtos deseja cadastrar (1 a 10)? ");
81     scanf("%d", &quantidade);
82
83     if (quantidade < 1 || quantidade > 10){
84         printf("Erro: quantidade inv lida.\n");
85         return 1;
86     }
87     // Aloca o din mica
88     produtos = malloc(quantidade * sizeof(struct Produto));
89
90     if (produtos == NULL){
91         printf("Erro ao alocar mem ria.\n");
92         return 1;
93     }
94     cadastrarProdutos(produtos, quantidade);
95     listarProdutos(produtos, quantidade);
96     exibirResumo(produtos, quantidade);
97     free(produtos);
98     produtos = NULL;
99     return 0;
100 }
101 }

```

Fonte: Autoria própria (2026).

5.2 Evidências de execução

Logo a seguir nas figuras 1 e 2, estão visíveis a execução das partes do dado código apresentado na subseção 5.1, elas são capturas de tela de um compilador online *online* chamado de *Programiz*. Elas mostram a execução do projeto como de acordo com o definido e seguindo os parâmetros organizados pelo professor responsável pela disciplina Douglas Lopes de Sousa Mendes. Na figura 1, é mostrado a parte do cadastro das figuras, e na 2 o resultado final que mostra os produtos cadastrados e o resumo do estoque.

Figura 1: Interface do compilador online mostrando o cadastro.

```
Output
-----
SISTEMA DE CADASTRO DE PRODUTOS
-----
Quantos produtos deseja cadastrar (1 a 10)? 3
-----
CADASTRO DO PRODUTO 1
-----
Código: 2321
Nome: Arroz branco
Preço: 6.44
Quantidade disponível: 5
-----
CADASTRO DO PRODUTO 2
-----
Código: 2322
Nome: Feijao preto
Preço: 5.22
Quantidade disponível: 6
-----
CADASTRO DO PRODUTO 3
-----
Código: 2323
Nome: Milho
Preço: 7.11
Quantidade disponível: 4
```

Fonte: Autoria própria (2026).

Figura 2: Interface do compilador online mostrando o resultado do processo.

```
-----  
PRODUTOS CADASTRADOS  
-----  
  
----- PRODUTO 1 -----  
Código: 2321  
Nome: Arroz branco  
Preço: R$ 6.44  
Quantidade: 5  
Valor em estoque: R$ 32.20  
  
----- PRODUTO 2 -----  
Código: 2322  
Nome: Feijao preto  
Preço: R$ 5.22  
Quantidade: 6  
Valor em estoque: R$ 31.32  
  
----- PRODUTO 3 -----  
Código: 2323  
Nome: Milho  
Preço: R$ 7.11  
Quantidade: 4  
Valor em estoque: R$ 28.44  
  
-----  
RESUMO DO ESTOQUE  
-----  
Valor total do estoque: R$ 91.96  
Produto mais caro: Milho  
Preço do produto mais caro: R$ 7.11  
  
=== Code Execution Successful ===
```

Fonte: Autoria própria (2026).

6 CONCLUSÃO

Durante o desenvolvimento da atividade, foi criado um sistema de controle de estoque em linguagem C, permitindo o cadastro, consulta e gerenciamento de produtos. Para sua construção, foram utilizados conceitos de programação estruturada, como estruturas , vetores, funções, entrada e saída de dados, além de bibliotecas básicas da linguagem. Além de desenvolvidos melhores noções da produção de relatórios acadêmicos seguindo as normas padrão da ABNT (Associação Brasileira de Normas

As principais dificuldades encontradas estiveram relacionadas com a implementação das funcionalidades de forma correta. Com a realização da atividade, o grupo ampliou seus conhecimentos sobre programação em C, aprendendo a aplicar conceitos teóricos vistos durante os estudos em sala, desenvolver a lógica de programação e estruturar melhor um sistema para resolver problemas do cotidiano relacionados ao controle de estoque.